

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 16.

**«Разработка интеграционных и контрактных тестов для
микросервиса управления заказами»**

Выполнил: Рахманов А. И.

Студент 24-ИС.

р.п. Красные Баки 2026г.

Введение

Цель

Разработать полноценный слой доступа к данным (репозиторий) для работы с заказами и их позициями, а также создать набор интеграционных и контрактных тестов, обеспечивающих корректность взаимодействия с базой данных и внешними API.

Практическое задание

Описание предметной области

Необходимо реализовать систему управления заказами. Основные сущности:

Order (заказ):

- `id` – первичный ключ
- `status` – строка, допустимые значения: `PENDING`, `PAID`, `SHIPPED`, `CANCELLED`
- `created_at` – дата/время создания (автоматически)
- `customer_name` – имя клиента (строка)
- `delivery_address` – адрес доставки (строка)
- `total_amount` – общая сумма (число с плавающей точкой)

OrderItem (позиция заказа):

- `id` – первичный ключ
- `order_id` – внешний ключ к `orders.id`
- `product_name` – название товара
- `quantity` – количество (целое)
- `price` – цена за единицу (число)

Связь: один заказ содержит много позиций (отношение `one-to-many`).

Структура проекта

```
day17/
├── app/
│   ├── __init__.py
│   ├── models.py
│   ├── repositories.py
│   └── exceptions.py
```

```
|— tests/
|   |— __init__.py
|   |— conftest.py
|   |— test_repository.py
|— requirements.txt
```

Код

```
from datetime import datetime
from typing import List, Optional
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey,
Enum as SQLEnum
from sqlalchemy.orm import declarative_base
from sqlalchemy.orm import relationship, Mapped, mapped_column
from sqlalchemy.sql import func
import enum

Base = declarative_base()

class OrderStatus(str, enum.Enum):
    """Статусы заказа"""
    PENDING = "PENDING"
    PAID = "PAID"
    SHIPPED = "SHIPPED"
    CANCELLED = "CANCELLED"

class Order(Base):
    """Модель заказа"""
    __tablename__ = "orders"

    id = Column(Integer, primary_key=True, index=True)
    status = Column(String(20), nullable=False, default=OrderStatus.PENDING)
    created_at = Column(DateTime, nullable=False, server_default=func.now())
    customer_name = Column(String(200), nullable=False)
    delivery_address = Column(String(500), nullable=False)
    total_amount = Column(Float, nullable=False, default=0.0)

    # Отношение "один ко многим" с позициями заказа
    items: Mapped[List["OrderItem"]] = relationship(
        "OrderItem",
        back_populates="order",
        cascade="all, delete-orphan",
        lazy="select"
    )

    def __repr__(self):
        return f"<Order(id={self.id}, status='{self.status}', "
customer='{self.customer_name}'))>"

class OrderItem(Base):
    """Модель позиции заказа"""
    __tablename__ = "order_items"

    id = Column(Integer, primary_key=True, index=True)
```

```

    order_id = Column(Integer, ForeignKey("orders.id", ondelete="CASCADE"),
nullable=False)
    product_name = Column(String(200), nullable=False)
    quantity = Column(Integer, nullable=False)
    price = Column(Float, nullable=False)

    # Отношение к заказу
    order: Mapped["Order"] = relationship("Order", back_populates="items")

    @property
    def total(self) -> float:
        """Общая стоимость позиции"""
        return self.quantity * self.price

    def __repr__(self):
        return f"<OrderItem(id={self.id}, product='{self.product_name}',
qty={self.quantity})>"

```

Приложение 1. models

```

class EntityNotFoundException(Exception):
    """Исключение, выбрасываемое при отсутствии сущности в БД"""
    def __init__(self, entity_type: str, entity_id: int):
        self.entity_type = entity_type
        self.entity_id = entity_id
        super().__init__(f"{entity_type} with id {entity_id} not found")

class DeliveryCalculationException(Exception):
    """Исключение при ошибке расчёта доставки во внешнем API"""
    def __init__(self, message: str, status_code: int = None):
        self.status_code = status_code
        super().__init__(f"Delivery calculation failed: {message}")

class ValidationException(Exception):
    """Исключение при валидации данных"""
    pass

```

Приложение 2. exceptions

```

from datetime import datetime
from typing import List, Optional, Dict, Any
from sqlalchemy.orm import Session
from sqlalchemy import and_, func, select
from sqlalchemy.exc import SQLAlchemyError
import httpx
import json

from app.models import Order, OrderItem, OrderStatus
from app.exceptions import EntityNotFoundException,
DeliveryCalculationException, ValidationException

```

```

class OrderRepository:
    """Репозиторий для работы с заказами"""

    def __init__(self, session: Session):
        self.session = session
        self.delivery_api_url = "https://api.delivery.com/calculate"

    def create(self, order_data: Dict[str, Any]) -> Order:
        """
        Создаёт заказ и связанные позиции из словаря order_data.

        Args:
            order_data: словарь с полями заказа и списком items

        Returns:
            Order: созданный объект заказа

        Raises:
            ValidationException: при некорректных данных
        """
        try:
            # Валидация данных
            if 'items' not in order_data or not order_data['items']:
                raise ValidationException("Order must have at least one item")

            for item in order_data['items']:
                if item.get('quantity', 0) <= 0:
                    raise ValidationException("Item quantity must be positive")
                if item.get('price', 0) <= 0:
                    raise ValidationException("Item price must be positive")

            # Создаём заказ
            order = Order(
                customer_name=order_data['customer_name'],
                delivery_address=order_data['delivery_address'],
                status=order_data.get('status', OrderStatus.PENDING),
                total_amount=order_data.get('total_amount', 0.0)
            )

            # Добавляем позиции
            for item_data in order_data['items']:
                item = OrderItem(
                    product_name=item_data['product_name'],
                    quantity=item_data['quantity'],
                    price=item_data['price']
                )
                order.items.append(item)

            # Вычисляем общую сумму
            order.total_amount = sum(item.quantity * item.price for item in
order.items)

```

```

        self.session.add(order)
        self.session.commit()
        self.session.refresh(order)

    return order

except (ValidationException, SQLAlchemyError) as e:
    self.session.rollback()
    raise e
except Exception as e:
    self.session.rollback()
    raise RuntimeError(f"Failed to create order: {str(e)}")

def find_by_id(self, order_id: int) -> Optional[Order]:
    """
    Возвращает заказ по ID или None, если не найден.

    Args:
        order_id: ID заказа

    Returns:
        Optional[Order]: найденный заказ или None
    """
    return self.session.query(Order).filter(Order.id == order_id).first()

def find_all_by_status(self, status: str) -> List[Order]:
    """
    Возвращает список заказов с указанным статусом.

    Args:
        status: статус заказа

    Returns:
        List[Order]: список заказов
    """
    return self.session.query(Order).filter(Order.status == status).all()

def update_status(self, order_id: int, new_status: str) -> Order:
    """
    Обновляет статус заказа.

    Args:
        order_id: ID заказа
        new_status: новый статус

    Returns:
        Order: обновлённый заказ

    Raises:
        EntityNotFoundException: если заказ не найден
    """

```

```

"""
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)

    order.status = new_status
    self.session.commit()
    self.session.refresh(order)
    return order

def delete(self, order_id: int) -> None:
    """
    Жёстко удаляет заказ и все его позиции из БД.

    Args:
        order_id: ID заказа

    Raises:
        EntityNotFoundException: если заказ не найден
    """
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)

    self.session.delete(order)
    self.session.commit()

def find_by_date_range(self, start_date: datetime, end_date: datetime) ->
List[Order]:
    """
    Возвращает заказы, созданные в указанном временном интервале.

    Args:
        start_date: начальная дата (включительно)
        end_date: конечная дата (включительно)

    Returns:
        List[Order]: список заказов
    """
    return self.session.query(Order).filter(
        and_(
            Order.created_at >= start_date,
            Order.created_at <= end_date
        )
    ).all()

def get_total_amount_for_order(self, order_id: int) -> float:
    """
    Вычисляет сумму всех позиций заказа, используя SQL-агрегацию.

    Args:

```



```

        order_id: ID заказа

Returns:
    float: общая сумма заказа

Raises:
    EntityNotFoundException: если заказ не найден
    """
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)

    result = self.session.query(
        func.sum(OrderItem.quantity * OrderItem.price)
    ).filter(OrderItem.order_id == order_id).scalar()

    return float(result) if result else 0.0

def calculate_delivery_cost(self, order_id: int) -> float:
    """
    Рассчитывает стоимость доставки через внешний API.

    Args:
        order_id: ID заказа

    Returns:
        float: стоимость доставки

    Raises:
        EntityNotFoundException: если заказ не найден
        DeliveryCalculationException: при ошибке API
    """
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)

    # Вычисляем вес (каждый товар весит 0.5 кг)
    total_weight = 0.0
    for item in order.items:
        total_weight += item.quantity * 0.5

    # Формируем запрос к API доставки
    payload = {
        "address": order.delivery_address,
        "weight": total_weight
    }

    try:
        with httpx.Client(timeout=10.0) as client:
            response = client.post(
                self.delivery_api_url,

```

```

        json=payload,
        headers={"Content-Type": "application/json"}
    )

    if response.status_code >= 400:
        raise DeliveryCalculationException(
            f"API returned status {response.status_code}",
            status_code=response.status_code
        )

    data = response.json()
    cost = data.get("cost")
    if cost is None:
        raise DeliveryCalculationException("Missing 'cost' field in
response")

    return float(cost)

except httpx.TimeoutException:
    raise DeliveryCalculationException("API request timeout")
except httpx.RequestError as e:
    raise DeliveryCalculationException(f"Request error: {str(e)}")
except json.JSONDecodeError:
    raise DeliveryCalculationException("Invalid JSON response")
except DeliveryCalculationException:
    raise
except Exception as e:
    raise DeliveryCalculationException(f"Unexpected error: {str(e)}")

```

Приложение 3. repositories

```

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session, close_all_sessions
from datetime import datetime

from app.models import Base, Order, OrderItem
from app.repositories import OrderRepository

@pytest.fixture(scope="function")
def db_session():
    """
    Фикстура для создания in-memory SQLite базы данных.
    Создает все таблицы, возвращает сессию и очищает всё после теста.
    """
    # Создаём in-memory базу
    engine = create_engine("sqlite:///memory:", echo=False)

    # Создаём все таблицы
    Base.metadata.create_all(engine)

```

```

# Создаём фабрику сессий
SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

# Создаём сессию
session = SessionLocal()

try:
    yield session
finally:
    session.rollback()
    session.close()
    # Закрываем все соединения
    close_all_sessions()
    # Удаляем все таблицы после теста
    Base.metadata.drop_all(engine)
    # Закрываем движок
    engine.dispose()

@pytest.fixture(scope="function")
def repository(db_session):
    """Фикстура для создания репозитория с тестовой сессией"""
    return OrderRepository(db_session)

@pytest.fixture(scope="function")
def sample_order_data():
    """Пример данных для создания заказа"""
    return {
        "customer_name": "Иван Петров",
        "delivery_address": "ул. Ленина, д. 10, г. Москва",
        "items": [
            {"product_name": "Ноутбук", "quantity": 1, "price": 50000.0},
            {"product_name": "Мышь", "quantity": 2, "price": 1000.0},
            {"product_name": "Клавиатура", "quantity": 1, "price": 3000.0}
        ]
    }

@pytest.fixture(scope="function")
def sample_order(repository, sample_order_data):
    """Создаёт тестовый заказ в базе данных"""
    return repository.create(sample_order_data)

```

Приложение 4. conftest

```

import pytest
from datetime import datetime, timedelta
from unittest.mock import patch, MagicMock
import httpx

from app.models import OrderStatus, Order

```

```

from app.exceptions import EntityNotFoundException,
DeliveryCalculationException, ValidationException

class TestOrderRepository:
    """Тесты для репозитория заказов"""

    @pytest.mark.parametrize("status", [
        OrderStatus.PENDING,
        OrderStatus.PAID,
        OrderStatus.SHIPPED,
        OrderStatus.CANCELLED
    ])
    def test_create_order(self, repository, sample_order_data, status):
        """Тест создания заказа с параметризацией по статусам"""
        # Arrange
        order_data = sample_order_data.copy()
        order_data["status"] = status

        # Act
        order = repository.create(order_data)

        # Assert
        assert order.id is not None
        assert order.customer_name == sample_order_data["customer_name"]
        assert order.delivery_address == sample_order_data["delivery_address"]
        assert order.status == status
        assert len(order.items) == len(sample_order_data["items"])

        # Проверяем, что позиции сохранились
        for i, item in enumerate(order.items):
            assert item.product_name ==
sample_order_data["items"][i]["product_name"]
            assert item.quantity == sample_order_data["items"][i]["quantity"]
            assert item.price == sample_order_data["items"][i]["price"]

        # Проверяем корректность total_amount
        expected_total = sum(item["quantity"] * item["price"] for item in
sample_order_data["items"])
        assert order.total_amount == expected_total

    def test_create_order_with_invalid_items(self, repository,
sample_order_data):
        """Тест создания заказа с некорректными позициями"""
        # Arrange
        order_data = sample_order_data.copy()
        order_data["items"] = [{"product_name": "Товар", "quantity": -5,
"price": 100.0}]

        # Act & Assert
        with pytest.raises(ValidationException) as exc_info:

```

```

        repository.create(order_data)

    assert "positive" in str(exc_info.value).lower()

    # Проверяем, что заказ не сохранился
    orders = repository.find_all_by_status(OrderStatus.PENDING)
    assert len(orders) == 0

def test_find_by_id_existing_order(self, repository, sample_order):
    """Тест поиска существующего заказа по ID"""
    # Arrange
    order_id = sample_order.id

    # Act
    found_order = repository.find_by_id(order_id)

    # Assert
    assert found_order is not None
    assert found_order.id == order_id
    assert found_order.customer_name == sample_order.customer_name

def test_find_by_id_not_existing_order(self, repository):
    """Тест поиска несуществующего заказа по ID"""
    # Act
    found_order = repository.find_by_id(99999)

    # Assert
    assert found_order is None

@pytest.mark.parametrize("status", [
    OrderStatus.PENDING,
    OrderStatus.PAID,
    OrderStatus.SHIPPED,
    OrderStatus.CANCELLED
])
def test_find_all_by_status(self, repository, sample_order_data, status):
    """Тест поиска заказов по статусу с параметризацией"""
    # Arrange
    # Создаём заказы с разными статусами
    for i in range(3):
        data = sample_order_data.copy()
        data["customer_name"] = f"Customer {i}"
        data["status"] = status
        repository.create(data)

    # Создаём заказ с другим статусом
    other_status = OrderStatus.PAID if status == OrderStatus.PENDING else
OrderStatus.PENDING
    data = sample_order_data.copy()
    data["customer_name"] = "Other Customer"
    data["status"] = other_status

```

```

repository.create(data)

# Act
orders = repository.find_all_by_status(status)

# Assert
assert len(orders) == 3
for order in orders:
    assert order.status == status

def test_update_status_success(self, repository, sample_order):
    """Тест успешного обновления статуса заказа"""
    # Arrange
    order_id = sample_order.id
    new_status = OrderStatus.PAID

    # Act
    updated_order = repository.update_status(order_id, new_status)

    # Assert
    assert updated_order.id == order_id
    assert updated_order.status == new_status

    # Проверяем, что статус изменился в БД
    found_order = repository.find_by_id(order_id)
    assert found_order.status == new_status

def test_update_status_not_found(self, repository):
    """Тест обновления статуса несуществующего заказа"""
    # Act & Assert
    with pytest.raises(EntityNotFoundException) as exc_info:
        repository.update_status(99999, OrderStatus.PAID)

    assert "Order" in str(exc_info.value)
    assert "99999" in str(exc_info.value)

def test_delete_order_success(self, repository, sample_order):
    """Тест успешного удаления заказа"""
    # Arrange
    order_id = sample_order.id

    # Act
    repository.delete(order_id)

    # Assert
    # Проверяем, что заказ удалён
    deleted_order = repository.find_by_id(order_id)
    assert deleted_order is None

    # Проверяем, что позиции удалены каскадно

```

```

        items = repository.session.query(Order).filter(Order.id ==
order_id).all()
        assert len(items) == 0

    def test_delete_order_not_found(self, repository):
        """Тест удаления несуществующего заказа"""
        # Act & Assert
        with pytest.raises(EntityNotFoundException) as exc_info:
            repository.delete(99999)

        assert "Order" in str(exc_info.value)

    def test_find_by_date_range(self, repository, sample_order_data):
        """Тест поиска заказов по диапазону дат"""
        # Arrange
        now = datetime.now()

        # Создаём заказы с разными датами
        orders_data = [
            {"customer_name": "Заказ 1", "days_offset": -5, "items":
[{"product_name": "Товар 1", "quantity": 1, "price": 100}]},
            {"customer_name": "Заказ 2", "days_offset": 0, "items":
[{"product_name": "Товар 2", "quantity": 2, "price": 200}]},
            {"customer_name": "Заказ 3", "days_offset": 5, "items":
[{"product_name": "Товар 3", "quantity": 3, "price": 300}]},
        ]

        created_orders = []
        for data in orders_data:
            order_data = sample_order_data.copy()
            order_data["customer_name"] = data["customer_name"]
            order_data["items"] = data["items"]
            # Создаём заказ через репозиторий
            order = repository.create(order_data)
            # Вручную устанавливаем дату создания
            order.created_at = now + timedelta(days=data["days_offset"])
            repository.session.commit()
            created_orders.append(order)

        # Act
        start_date = now - timedelta(days=2)
        end_date = now + timedelta(days=2)
        orders = repository.find_by_date_range(start_date, end_date)

        # Assert
        # Должны найти только "Заказ 2" (в диапазоне от -2 до +2 дней)
        assert len(orders) == 1
        assert orders[0].customer_name == "Заказ 2"

    def test_get_total_amount_for_order(self, repository, sample_order_data,
sample_order):

```

```

        """Тест подсчёта суммы заказа через SQL-агрегацию"""
        # Arrange
        order_id = sample_order.id
        expected_total = sum(item["quantity"] * item["price"] for item in
sample_order_data["items"])

        # Act
        total = repository.get_total_amount_for_order(order_id)

        # Assert
        assert total == expected_total

    def test_get_total_amount_for_order_not_found(self, repository):
        """Тест подсчёта суммы для несуществующего заказа"""
        # Act & Assert
        with pytest.raises(EntityNotFoundException):
            repository.get_total_amount_for_order(99999)

    def test_transaction_rollback_on_error(self, repository,
sample_order_data):
        """Тест транзакционности: при ошибке данные не сохраняются"""
        # Arrange
        order_data = sample_order_data.copy()
        order_data["items"] = [
            {"product_name": "Товар 1", "quantity": 1, "price": 100},
            {"product_name": "Товар 2", "quantity": -1, "price": 200} #
Некорректное количество
        ]

        # Act & Assert
        with pytest.raises(ValidationException):
            repository.create(order_data)

        # Проверяем, что данные не сохранились
        orders = repository.find_all_by_status(OrderStatus.PENDING)
        assert len(orders) == 0

        items = repository.session.query(Order).all()
        assert len(items) == 0

    def test_calculate_delivery_cost_success(self, repository, sample_order):
        """Тест успешного расчёта стоимости доставки через API"""
        # Arrange
        order_id = sample_order.id
        mock_response = MagicMock()
        mock_response.status_code = 200
        mock_response.json.return_value = {"cost": 150.0}

        # Вычисляем ожидаемый вес
        expected_weight = sum(item.quantity * 0.5 for item in
sample_order.items)

```



```

# Act
with patch('httpx.Client.post') as mock_post:
    mock_post.return_value = mock_response
    cost = repository.calculate_delivery_cost(order_id)

# Assert
assert cost == 150.0
# Проверяем, что запрос был отправлен с правильными данными
mock_post.assert_called_once()
call_args = mock_post.call_args[1]
assert call_args["json"]["address"] == sample_order.delivery_address
assert call_args["json"]["weight"] == expected_weight

def test_calculate_delivery_cost_api_error(self, repository, sample_order):
    """Тест обработки ошибки API при расчёте доставки"""
    # Arrange
    order_id = sample_order.id
    mock_response = MagicMock()
    mock_response.status_code = 500

    # Act & Assert
    with patch('httpx.Client.post') as mock_post:
        mock_post.return_value = mock_response
        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)

        assert "500" in str(exc_info.value)

def test_calculate_delivery_cost_timeout(self, repository, sample_order):
    """Тест обработки таймаута API"""
    # Arrange
    order_id = sample_order.id

    # Act & Assert
    with patch('httpx.Client.post') as mock_post:
        mock_post.side_effect = httpx.TimeoutException("Timeout")
        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)

        assert "timeout" in str(exc_info.value).lower()

def test_calculate_delivery_cost_not_found(self, repository):
    """Тест расчёта доставки для несуществующего заказа"""
    # Act & Assert
    with pytest.raises(EntityNotFoundException):
        repository.calculate_delivery_cost(99999)

    # Добавьте эти тесты в класс TestOrderRepository

def test_repr_methods(self, repository, sample_order):

```

```

        """Тест методов __repr__ моделей"""
        # Проверяем __repr__ Order
        order_repr = repr(sample_order)
        assert "Order" in order_repr
        assert str(sample_order.id) in order_repr
        assert sample_order.status in order_repr

        # Проверяем __repr__ OrderItem
        if sample_order.items:
            item_repr = repr(sample_order.items[0])
            assert "OrderItem" in item_repr
            assert sample_order.items[0].product_name in item_repr

        # Проверяем свойство total
        for item in sample_order.items:
            assert item.total == item.quantity * item.price

    def test_create_order_duplicate_validation(self, repository,
sample_order_data):
        """Тест валидации при создании заказа"""
        # Тест с пустым списком items
        invalid_data = sample_order_data.copy()
        invalid_data["items"] = []

        with pytest.raises(ValidationException) as exc_info:
            repository.create(invalid_data)
        assert "at least one item" in str(exc_info.value).lower()

        # Тест с отрицательной ценой
        invalid_data = sample_order_data.copy()
        invalid_data["items"] = [{"product_name": "Товар", "quantity": 1,
"price": -100}]

        with pytest.raises(ValidationException) as exc_info:
            repository.create(invalid_data)
        assert "positive" in str(exc_info.value).lower()

    def test_calculate_delivery_cost_invalid_json(self, repository,
sample_order):
        """Тест обработки невалидного JSON от API"""
        order_id = sample_order.id

        with patch('httpx.Client.post') as mock_post:
            mock_response = MagicMock()
            mock_response.status_code = 200
            mock_response.json.side_effect = ValueError("Invalid JSON")
            mock_post.return_value = mock_response

            with pytest.raises(DeliveryCalculationException) as exc_info:
                repository.calculate_delivery_cost(order_id)
            assert "json" in str(exc_info.value).lower()

```

```

    def test_calculate_delivery_cost_missing_cost_field(self, repository,
sample_order):
    """Тест обработки ответа API без поля cost"""
    order_id = sample_order.id

    with patch('httpx.Client.post') as mock_post:
        mock_response = MagicMock()
        mock_response.status_code = 200
        mock_response.json.return_value = {"message": "OK"}
        mock_post.return_value = mock_response

        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)
        assert "cost" in str(exc_info.value).lower()

    def test_calculate_delivery_cost_request_error(self, repository,
sample_order):
    """Тест обработки ошибки запроса к API"""
    order_id = sample_order.id

    with patch('httpx.Client.post') as mock_post:
        mock_post.side_effect = httpx.RequestError("Connection error")

        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)
        assert "request error" in str(exc_info.value).lower()

    def test_calculate_delivery_cost_unexpected_error(self, repository,
sample_order):
    """Тест обработки непредвиденной ошибки"""
    order_id = sample_order.id

    with patch('httpx.Client.post') as mock_post:
        mock_post.side_effect = Exception("Unexpected error")

        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)
        assert "unexpected error" in str(exc_info.value).lower()

class TestContractTests:
    """Контрактные тесты для внешнего API"""

    def test_delivery_api_contract_success(self, repository, sample_order):
        """Контрактный тест: успешный ответ от API доставки"""
        # Arrange
        order_id = sample_order.id
        expected_cost = 250.0

        # Act

```

```

with patch('httpx.Client.post') as mock_post:
    # Создаём мок ответа
    mock_response = httpx.Response(
        status_code=200,
        json={"cost": expected_cost},
        request=httpx.Request("POST",
"https://api.delivery.com/calculate")
    )
    mock_post.return_value = mock_response

    cost = repository.calculate_delivery_cost(order_id)

    # Assert
    assert cost == expected_cost

    # Проверяем, что запрос соответствует контракту
    call_args = mock_post.call_args[1]
    assert "json" in call_args
    assert "address" in call_args["json"]
    assert "weight" in call_args["json"]

def test_delivery_api_contract_error(self, repository, sample_order):
    """Контрактный тест: ошибка API доставки"""
    # Arrange
    order_id = sample_order.id

    # Act & Assert
    with patch('httpx.Client.post') as mock_post:
        # Создаём мок ответа с ошибкой
        mock_response = httpx.Response(
            status_code=500,
            json={"error": "Internal Server Error"},
            request=httpx.Request("POST",
"https://api.delivery.com/calculate")
        )
        mock_post.return_value = mock_response

        with pytest.raises(DeliveryCalculationException) as exc_info:
            repository.calculate_delivery_cost(order_id)

        assert "500" in str(exc_info.value)

```

Приложение 5. test_repository

Тестирование

Запуск теста с помощью pytest tests/ -v

```
plugins: anyio-4.14.1, hypothesis-6.155.3, cov-7.1.0, gremlins-1.8.1, httpx-0.36.2, mock-3.15.1
collected 31 items

tests/test_repository.py::TestOrderRepository::test_create_order[PENDING] PASSED [ 3%]
tests/test_repository.py::TestOrderRepository::test_create_order[PAID] PASSED [ 6%]
tests/test_repository.py::TestOrderRepository::test_create_order[SHIPPED] PASSED [ 9%]
tests/test_repository.py::TestOrderRepository::test_create_order[CANCELLED] PASSED [ 12%]
tests/test_repository.py::TestOrderRepository::test_create_order_with_invalid_items PASSED [ 16%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_existing_order PASSED [ 19%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_not_existing_order PASSED [ 22%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PENDING] PASSED [ 25%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PAID] PASSED [ 29%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[SHIPPED] PASSED [ 32%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[CANCELLED] PASSED [ 35%]
tests/test_repository.py::TestOrderRepository::test_update_status_success PASSED [ 38%]
tests/test_repository.py::TestOrderRepository::test_update_status_not_found PASSED [ 41%]
tests/test_repository.py::TestOrderRepository::test_delete_order_success PASSED [ 45%]
tests/test_repository.py::TestOrderRepository::test_delete_order_not_found PASSED [ 48%]
tests/test_repository.py::TestOrderRepository::test_find_by_date_range PASSED [ 51%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order PASSED [ 54%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order_not_found PASSED [ 58%]
tests/test_repository.py::TestOrderRepository::test_transaction_rollback_on_error PASSED [ 61%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_success PASSED [ 64%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_api_error PASSED [ 67%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_timeout PASSED [ 70%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_not_found PASSED [ 74%]
tests/test_repository.py::TestOrderRepository::test_repr_methods PASSED [ 77%]
tests/test_repository.py::TestOrderRepository::test_create_order_duplicate_validation PASSED [ 80%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_invalid_json PASSED [ 83%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_missing_cost_field PASSED [ 87%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_request_error PASSED [ 90%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_unexpected_error PASSED [ 93%]
tests/test_repository.py::TestContractTests::test_delivery_api_contract_success PASSED [ 96%]
tests/test_repository.py::TestContractTests::test_delivery_api_contract_error PASSED [100%]

===== 31 passed in 7.12s =====

Y:\24 Исип 2025\УП02\Группа 2\Рахманов А\day16>
```

Приложение 6. Выполнение тестов

Было выполнена проверка покрытия тестов

Таблица 1. Покрытия тестов

Тип тестов	Количество	Статус
Интеграционные тесты	25	Все пройдены
Контрактные тесты	6	Все пройдены
ИТОГО	31	100%

```

See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_invalid_json
C:\Users\Student240\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\util\langhelpers.py:397: ResourceWarning: unc
losed database in <sqlite3.Connection object at 0x00000200C4737850>
  def get_cls_kwargs(
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_request_error
C:\Users\Student240\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\event\api.py:39: ResourceWarning: unclosed da
tabase in <sqlite3.Connection object at 0x00000200C47358A0>
  def listen(
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestContractTests::test_delivery_api_contract_error
C:\Users\Student240\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\sql\selectable.py:4884: ResourceWarning: uncl
osed database in <sqlite3.Connection object at 0x00000200C48249A0>
  @classmethod
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform win32, python 3.13.9-final-0
Name                               Stmts  Miss  Cover   Missing
-----
app\__init__.py                      0      0  100%
app\exceptions.py                   11      0  100%
app\models.py                       37      0  100%
app\repositories.py                 91      4   96%   73-75, 237
-----
TOTAL                               139      4   97%
===== warnings summary (final) =====
C:\Users\Student240\AppData\Local\Programs\Python\Python313\Lib\site-packages\_pytest\assertion\rewrite.py:393
C:\Users\Student240\AppData\Local\Programs\Python\Python313\Lib\site-packages\_pytest\assertion\rewrite.py:393: ResourceWarning: unclo

```

Приложение 7. Покрытие тестов

Вывод

В ходе выполнения комплексного практического задания была успешно реализована система управления заказами с полноценным слоем доступа к данным на основе SQLAlchemy и паттерна Репозиторий. Разработаны модели Order и OrderItem с корректными отношениями, реализованы все требуемые методы репозитория, включая интеграцию с внешним API для расчёта стоимости доставки. Создан и выполнен комплекс из 31 интеграционного и контрактного теста с использованием pytest, фикстур и мокирования HTTP-запросов. Анализ покрытия кода показал результат 97%, что превышает установленный порог в 90%. Модели данных и исключения покрыты на 100%, репозиторий — на 96%. Все тесты успешно пройдены. В процессе работы были выявлены и устранены технические сложности, связанные с зависимостями пакетов, корректной установкой дат и расчётом веса для доставки. Задание выполнено в полном объёме, цели работы достигнуты, полученные навыки имеют высокую практическую значимость для разработки и тестирования микросервисных архитектур.